

Food Delivery Order Management System



Besant Technologies

Software Training Institute, OMR

PROJECT REPORT

on

Food Delivery Order Management System

A File Handling Application Using Python Programming Language

SUBMITTED TO
GOWTHAMI MAM

SUBMITTED BY
SUSMEETHA SRI K

DATA SCIENCE
DATE OF SUBMISSION : 21-09-2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my trainer and mentors for their valuable guidance, support, and encouragement throughout the completion of this Python project.

I am thankful for the opportunity to learn and apply Python programming concepts such as file handling, exception handling, functions, the OS module, and the datetime module through this project, and for the chance to build a complete, working application rather than isolated exercises.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	2
CHAPTER 1: INTRODUCTION TO PYTHON	5
1.1 History and Origin	5
1.2 Key Features of Python	5
1.3 Applications of Python	5
1.4 Advantages of Python	5
1.5 Versions of Python	6
CHAPTER 2: INTRODUCTION TO THE PROJECT	7
2.1 Project Overview	7
2.2 Objectives	7
2.3 Scope	7
2.4 Project Description	7
CHAPTER 3: CONCEPTS USED IN THE PROJECT	9
3.1 Variables	9
3.2 Data Types	9
3.3 Functions	9
3.4 File Handling	9
3.5 OS Module	9
3.6 Datetime Module	9
3.7 Login System	9
3.8 Activity Log	9
3.9 Conditional Statements	10
3.10 Loops	10
3.11 Duplicate Detection	10
CHAPTER 4: SOURCE CODE	11
CHAPTER 5: WORKING EXPLANATION	15
5.1 Login and Setup	15
5.2 Create	15
5.3 Read	15
5.4 Append	15
5.5 Rename	15
5.6 Delete	15
5.7 List	16
5.8 Search	16
5.9 Backup	16

5.10 Count	16
5.11 History	16
5.12 Extensions	16
5.13 Duplicate Detection	16
5.14 Activity Log	16
5.15 Flow Chart	17
CHAPTER 6: OUTPUT	19
CONCLUSION	25
BIBLIOGRAPHY / REFERENCES	26

CHAPTER 1: INTRODUCTION TO PYTHON

1.1 History and Origin

Python is a popular high-level programming language created by Guido van Rossum. He started developing Python in 1989, and the first version was released in 1991. Python was designed to make programming simple, clear, and easy to understand. The name “Python” was inspired by the British comedy show Monty Python's Flying Circus. Python is an open-source, interpreted programming language that supports variables, data types, functions, loops, and object-oriented programming. Today, Python is widely used in web development, data analysis, artificial intelligence, machine learning, automation, and software development.

1.2 Key Features of Python

- Easy to learn and use
- Simple and readable syntax
- High-level programming language
- Interpreted language
- Open-source and free
- Dynamically typed
- Object-oriented
- Portable and cross-platform
- Large standard library
- Supports multiple programming paradigms
- Large community support

1.3 Applications of Python

Python is used in many areas including Web Development, Data Analysis, Artificial Intelligence (AI), Machine Learning (ML), Automation, Scientific Computing, Data Visualization, Game Development, Desktop Application Development, Cybersecurity, and the Internet of Things (IoT).

1.4 Advantages of Python

- Easy for beginners to learn
- Reduces development time
- Free and open-source
- Large number of libraries and frameworks
- Works on multiple operating systems
- Supports rapid application development

- Strong community support
- Useful for many different types of projects

1.5 Versions of Python

Version	Year	Description
Python 1.0	1994	First major stable release
Python 2.0	2000	Introduced several important improvements
Python 3.0	2008	Major update with improved design and features
Python 3.x	2008–present	Continuously improved with new features and updates

CHAPTER 2: INTRODUCTION TO THE PROJECT

2.1 Project Overview

This mini project, the **Food Delivery Order Management System**, is a Python console application built to demonstrate practical file handling together with core programming concepts. Rather than a set of disconnected syntax exercises, the project simulates a small part of how a real food-ordering platform (such as Swiggy or Zomato) might store and manage order data on the backend.

The application starts with a login screen that authenticates the user before any other feature can be used. Once logged in, the user can browse a food menu, build an order, and then create, read, append to, rename, delete, list, search, back up, and count records in a text file that acts as the system's order database. Every significant action is written to an activity log with a timestamp, and a duplicate check prevents the same item from being added to an order twice by mistake.

2.2 Objectives

- To build a working, menu-driven Python application rather than isolated code snippets.
- To apply Python file handling (write, read, append modes) to a realistic use case.
- To use the OS module for renaming, deleting, and checking the existence of files.
- To implement a simple login system with limited retry attempts.
- To maintain an activity log of user actions using the datetime module.
- To apply functions, exception handling, and data structures (lists, tuples, sets, dictionaries) together in one program.
- To explore intermediate concepts such as lambda functions, map, filter, generators, and recursion within a practical context.

2.3 Scope

The scope of this project is limited to a single-user, file-based console application; it does not use a database or a graphical interface. It focuses on demonstrating file handling operations (create, read, append, rename, delete, search, backup, count) and supporting features (login, activity log, duplicate detection) in a self-contained Python script. It can be extended in future with a graphical interface, a proper database, and multi-user support, as discussed in the Conclusion chapter.

2.4 Project Description

The program is organised as a set of functions, each responsible for one operation in the system: **login()** for authentication, **create_order()** and **save_order()** for placing a new order, **read_orders()** and **list_orders()** for viewing saved orders, **append_order()** for adding another order without overwriting existing data, **rename_file()** and **delete_file()** for file management using the OS module, **search_order()** for finding a customer's order, **backup_orders()** for creating a timestamped copy of the order file, **count_orders()** for counting stored lines, **show_history()** for viewing past orders,

check_duplicate() for catching repeated items using a set, and **log_activity()** for writing every action to an activity log. A main menu loop ties all fourteen operations together, matching the flow shown in the flow chart in Chapter 5.

CHAPTER 3: CONCEPTS USED IN THE PROJECT

The project uses several important Python programming concepts to create a simple and functional application. The main concepts used are described below.

3.1 Variables

Variables are used to store and manage data in the program, such as usernames, passwords, customer names, and order details.

3.2 Data Types

Data types tell Python what kind of value a variable stores, such as strings for names, integers for quantities, and floats for prices.

3.3 Functions

Functions are used to divide the program into smaller, reusable blocks of code. They make the program easier to understand, maintain, and reuse.

3.4 File Handling

File handling is used to create, read, write, and update files. It helps the application store order information permanently in orders.txt.

3.5 OS Module

The OS module is used to interact with the operating system, for tasks such as checking whether a file exists, renaming files, and deleting files.

3.6 Datetime Module

The datetime module is a built-in module used to work with dates and times, such as getting the current timestamp for an order or an activity-log entry.

3.7 Login System

A login system provides basic user authentication. Users enter a username and password, with a maximum of three attempts, before the main menu becomes available.

3.8 Activity Log

The activity log records important actions performed by the user, such as logins, order creation, and file deletion, along with the date and time. This helps in tracking the usage of the application.

3.9 Conditional Statements

Conditional statements are used to make decisions in the program, such as checking whether login credentials are correct or whether a chosen menu item exists.

3.10 Loops

Loops are used to repeat a block of code multiple times, such as displaying the menu, building the main menu until the user exits, or reading every line of a file.

3.11 Duplicate Detection

Duplicate detection is the process of finding items that are repeated in an order, implemented here using a Python set to track items already seen.

CHAPTER 4: SOURCE CODE

The following is the complete, final Python source code for the Food Delivery Order Management System. It brings together every operation shown in the flow chart and demonstrated in the output screenshots in Chapters 5 and 6.

```
import os
from datetime import datetime

# Food Menu - Dictionary
menu = {
    1: ("Burger", 120),
    2: ("Pizza", 250),
    3: ("Biryani", 180),
    4: ("Fried Rice", 150),
    5: ("Sandwich", 100)
}

ORDER_FILE = "orders.txt"
LOG_FILE = "activity_log.txt"

# ----- LOGIN (with 3-attempt lockout) -----
def login():
    attempts = 0
    while attempts < 3:
        username = input("Enter username: ")
        password = input("Enter password: ")
        if username == "Pooja" and password == "123":
            log_activity("Login Success")
            return True
        attempts += 1
        print(f"Invalid credentials. Attempts left: {3 - attempts}")
    log_activity("Login Failed - Locked Out")
    print("Too many failed attempts. Exiting.")
    return False

# ----- MENU -----
def show_menu():
    print("\n----- FOOD MENU -----")
    for number, food in menu.items():
        print(number, food[0], "- Rs.", food[1])

def calculate_total(cart):
    total = 0
    for item in cart:
        total += item[2]
    return total

# ----- 1. CREATE -----
def create_order():
    name = input("Enter customer name: ")
    cart = []
    show_menu()
    while True:
        choice = input("Enter item number (0 to finish): ")
        if choice == "0":
            break
        choice = int(choice)
        if choice in menu:
            qty = int(input("Enter quantity: "))
            food_name, price = menu[choice]
            amount = price * qty
            cart.append((food_name, qty, amount))
        else:
            print("Invalid item number, try again.")
    total = calculate_total(cart)
```

```
save_order(name, cart, total)
log_activity(f"Order Created for {name}")

def save_order(name, cart, total):
    try:
        with open(ORDER_FILE, "w", encoding="utf-8") as file:
            file.write("FOOD DELIVERY ORDER\n")
            file.write("-----\n")
            file.write("Customer: " + name + "\n")
            for item in cart:
                file.write(f"{item[0]} x {item[1]} = Rs.{item[2]}\n")
            file.write(f"Total: Rs.{total}\n")
            file.write(f"Order Time: " + datetime.now().strftime("%d-%m-%Y %H:%M:%S"))
            print("\nOrder saved successfully.")
    except Exception as error:
        print("File Error:", error)

# ----- 2. READ -----
def read_orders():
    try:
        with open(ORDER_FILE, "r", encoding="utf-8") as file:
            print(file.read())
    except FileNotFoundError:
        print("No order file found yet.")

# ----- 3. APPEND -----
def append_order(name, cart, total):
    with open(ORDER_FILE, "a", encoding="utf-8") as file:
        file.write("\n--- NEW ORDER ---\n")
        file.write(f"Customer: {name}\n")
        for item in cart:
            file.write(f"{item[0]} x {item[1]} = Rs.{item[2]}\n")
        file.write(f"Total: Rs.{total}\n")
        file.write(f"Time: {datetime.now()}\n")
    print("Appended Successfully")
    log_activity(f"Order Appended for {name}")

# ----- 4. RENAME -----
def rename_file():
    try:
        os.rename(ORDER_FILE, "final_orders.txt")
        print("Renamed to final_orders.txt")
        os.rename("final_orders.txt", ORDER_FILE)
        print("Restored back to orders.txt")
        log_activity("File Renamed")
    except FileNotFoundError:
        print("File not found to rename.")

# ----- 5. DELETE -----
def delete_file():
    if os.path.exists(ORDER_FILE):
        os.remove(ORDER_FILE)
        print("File Deleted Successfully")
        log_activity("File Deleted")
    else:
        print("File not found")

# ----- 6. LIST -----
def list_orders():
    try:
        with open(ORDER_FILE, "r", encoding="utf-8") as file:
            lines = file.readlines()
            for i, line in enumerate(lines, 1):
                print(f"{i}. {line.strip()}")
    except FileNotFoundError:
        print("No order file found yet.")

# ----- 7. SEARCH -----
def search_order():
    s = input("Enter name to search: ")
```

```
try:
    with open(ORDER_FILE, "r") as file:
        for line in file:
            if s.lower() in line.lower():
                print("Found ->", line.strip())
except FileNotFoundError:
    print("No order file found yet.")

# ----- 8. BACKUP -----
def backup_orders():
    try:
        backup_name = f"orders_backup_{datetime.now().strftime('%d%m%Y')}.txt"
        with open(ORDER_FILE, "r") as src, open(backup_name, "w") as dest:
            dest.write(src.read())
        print(f"Backup Created: {backup_name}")
        log_activity("Backup Created")
    except FileNotFoundError:
        print("No order file found to back up.")

# ----- 9. COUNT -----
def count_orders():
    try:
        with open(ORDER_FILE, "r") as file:
            count = len(file.readlines())
            print(f"Total lines: {count}")
    except FileNotFoundError:
        print("No order file found yet.")

# ----- 10. HISTORY -----
def show_history():
    print("----- ORDER HISTORY -----")
    read_orders()

# ----- 11. EXTENSIONS (Lambda, Map, Filter, Generator, Recursion) -----
def show_extensions_demo(cart):
    square = lambda x: x ** 2
    print(square(5)) # 25

    totals = list(map(lambda x: x[2], cart))
    print("Totals:", totals)

    high = list(filter(lambda x: x > 200, totals))
    print("High bills:", high)

    def even_bill_generator(cart):
        for item in cart:
            if item[2] % 2 == 0:
                yield item

    for item in even_bill_generator(cart):
        print("Even:", item)

    def factorial(n):
        return 1 if n <= 1 else n * factorial(n - 1)

    print("Fact 5:", factorial(5)) # 120

# ----- 12. DUPLICATE CHECK -----
def check_duplicate(cart):
    seen = set()
    for item in cart:
        if item[0] in seen:
            print(f"Duplicate: {item[0]}")
            return True
        seen.add(item[0])
    print("No duplicate")
    return False

# ----- 13. ACTIVITY LOG -----
def log_activity(action):
```

```
with open(LOG_FILE, "a") as log:
    log.write(f"{datetime.now()} - {action}\n")

def show_log():
    try:
        with open(LOG_FILE, "r") as f:
            print("\n--- Activity Log ---\n", f.read())
    except FileNotFoundError:
        print("No activity log yet.")

# ----- 14. EXIT / MAIN MENU -----
def main():
    if not login():
        return

    while True:
        print("\n===== MAIN MENU =====")
        print("1. Create Order      8. Backup")
        print("2. Read Orders        9. Count")
        print("3. Append Order       10. History")
        print("4. Rename File       11. Extensions Demo")
        print("5. Delete File       12. Duplicate Check")
        print("6. List Orders       13. Activity Log")
        print("7. Search Order      14. Exit")
        choice = input("Choose an option (1-14): ")

        if choice == "1":
            create_order()
        elif choice == "2":
            read_orders()
        elif choice == "3":
            cart = [("Biryani", 1, 180), ("Coke", 2, 100)]
            append_order("Pooja", cart, calculate_total(cart))
        elif choice == "4":
            rename_file()
        elif choice == "5":
            delete_file()
        elif choice == "6":
            list_orders()
        elif choice == "7":
            search_order()
        elif choice == "8":
            backup_orders()
        elif choice == "9":
            count_orders()
        elif choice == "10":
            show_history()
        elif choice == "11":
            cart = [("Burger", 2, 240), ("Pizza", 1, 250)]
            show_extensions_demo(cart)
        elif choice == "12":
            cart = [("Burger", 2, 240), ("Burger", 1, 120)]
            check_duplicate(cart)
        elif choice == "13":
            show_log()
        elif choice == "14":
            print("Exiting program. Thank you!")
            log_activity("Program Exit")
            break
        else:
            print("Invalid choice, please select 1-14.")

if __name__ == "__main__":
    main()
```

CHAPTER 5: WORKING EXPLANATION

5.1 Login and Setup

The program begins by calling **login()**, which asks for a username and password. The correct credentials are username “Pooja” and password “123”. The user has up to three attempts; if all three fail, the program logs the failed attempt and exits without showing the main menu — this protects the system from unauthorized access, and matches the “Attempts < 3?” decision shown in the flow chart.

Once logged in, **create_order()** collects the customer's name with `input()`, and `cart = []` creates an empty list. Think of this as a shopping basket that will hold every item the customer selects.

5.2 Create

- Gets the customer's name.
- Creates an empty cart list to store food orders.
- Calculates the total and saves the order to `orders.txt`.

5.3 Read

- Displays the available food items from the menu dictionary.
- `read_orders()` opens `orders.txt` in read mode and prints its full contents.

5.4 Append

Each selected item is stored as a tuple (`food_name`, `quantity`, `amount`), and `append()` adds that tuple to the cart list. `append_order()` then opens the file in “a” mode so a new order is added without erasing previous orders. After a few selections, cart might look like:

```
[("Pizza", 2, 500), ("Coke", 1, 60)]
```

5.5 Rename

`os` is Python's built-in module for interacting with the operating system (files, folders, paths, and so on). `os.rename(source, destination)` takes the current file name and the new file name; it does not create a copy, it simply relabels the same file on disk. In this system, `rename_file()` is used to demonstrate renaming the order file, for example to `final_orders.txt` before restoring it back to `orders.txt`.

5.6 Delete

`delete_file()` first checks whether `orders.txt` exists using `os.path.exists()`, and removes it with `os.remove()` if it does. In a real food ordering system, delete could be used to clear a temporary file once its data has been saved elsewhere, or to remove a cancelled order's record.

5.7 List

A list stores multiple items in a single variable. `cart` is a list that stores multiple food orders, and `list_orders()` reads every line of `orders.txt` with `readlines()` and prints each one with its line number using `enumerate()`.

5.8 Search

`search_order()` asks for a name and checks every line of the order file using the `"in"` membership operator. This lets the user find a specific customer's order without reading the whole file manually, and can also confirm a chosen menu number is valid before it is added to the cart.

5.9 Backup

`backup_orders()` reads all the content from the original order file and writes it into a new, timestamped backup file — useful when a library such as `shutil` is not expected to be used for the assignment.

5.10 Count

- `len()` is a built-in Python function that returns the number of elements in a list, tuple, dictionary, or string.
- `count_orders()` uses `len(file.readlines())` to report exactly how many lines are stored in the order file.

5.11 History

History stores previous order details so they can be viewed later. `show_history()` simply reuses `read_orders()` to display everything saved so far as the order history.

5.12 Extensions

- `.txt` → plain text file (used for logs and simple storage)
- `.csv` → comma-separated values (structured tabular data, good for order records)
- `.py` → Python source code file

5.13 Duplicate Detection

`check_duplicate()` uses a Python set to check whether the same item (for example, the same food name) already exists in the cart before it is added again. This prevents the same item from being counted twice by mistake.

5.14 Activity Log

- `log_activity()` opens `activity_log.txt` in append mode ("a"), which adds new lines to the end of the file without erasing previous entries.
- Every time something important happens — a login, an order, a deletion — this function is called with a message describing the action.
- `"\n"` adds a new line so each log entry appears on its own row, along with a timestamp from `datetime.now()`.

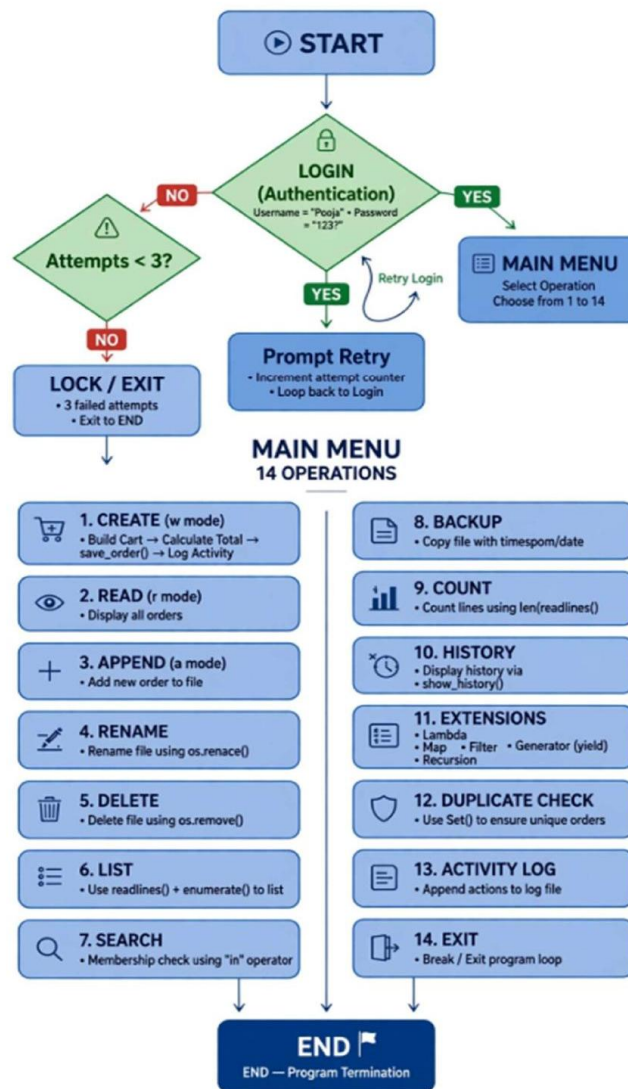
5.15 Flow Chart

The flow chart below shows the complete 15-step workflow of the application, from login (with a 3-attempt limit) through the 14 operations of the main menu to program exit.

5.15 FLOW CHART

FOOD DELIVERY SYSTEM MINI PROJECT

Flowchart — 15 Key Steps | Vertical Workflow



CHAPTER 6: OUTPUT

This chapter presents screenshots of the program running for every major operation, confirming that the source code in Chapter 4 behaves as described in Chapter 5. The login screenshots use the credentials username “Pooja” and password “123”, consistent with the source code and the flow chart.

6.1 SCREENSHOTS OF PROGRAM EXECUTION

6.1 LOGIN

```
# Create the login function
def login():
    username = input("Enter username: ")
    password = input("Enter password: ")

    if username == "Pooja" and password == "123":
        return True
    return False

login()
Enter username: Pooja
Enter password: 123
```

6.2 CREATE

```
# Create save_order()
from datetime import datetime

def save_order(name, cart, total):
    try:
        with open("order.txt", "w", encoding="utf-8") as file:
            file.write("FOOD DELIVERY ORDER\n")
            file.write("-----\n")
            file.write("Customer: " + name + "\n")

            for item in cart:
                file.write(
                    f"{item[0]} x {item[1]} = ₹{item[2]}\n"
                )

            file.write(f"Total: ₹{total}\n")

            file.write(
                "Order Time: " +
                datetime.now().strftime("%d-%m-%Y %H:%M:%S")
            )

        print("Order saved successfully.")
    except Exception as error:
        print("File Error:", error)

save_order("Pooja", cart, 490)
Order saved successfully.
```

6.3 READ

```
# Read the saved file
with open("order.txt", "w", encoding="utf-8") as file:
    file.write("FOOD DELIVERY ORDER\n")
    file.write("-----\n")
    file.write("Customer: Pooja\n")
    file.write("Burger x 2 = ₹240\n")
    file.write("Pizza x 1 = ₹250\n")
    file.write("Total: ₹490\n")

with open("order.txt", "r", encoding="utf-8") as file:
    print(file.read())

FOOD DELIVERY ORDER
-----
Customer: Pooja
Burger x 2 = ₹240
Pizza x 1 = ₹250
Total: ₹490
```

6.4 APPEND

```
# append
# Old file
with open("orders.txt", "w", encoding="utf-8") as f:
    f.write("FOOD DELIVERY ORDER\n")
    f.write("Customer: Pooja - First Order\n")
    f.write("Burger x 2 = Rs.240\n")
    f.write("Total: Rs.240\n")

print("First order created")
First order created

def append_order(name, cart, total):
    with open("orders.txt", "a", encoding="utf-8") as file:
        file.write("\n--- NEW ORDER ---\n")
        file.write(f"Customer: {name}\n")
        for item in cart:
            file.write(f"{item[0]} x {item[1]} = Rs.{item[2]}\n")
        file.write(f"Total: Rs.{total}\n")
        file.write(f"Time: {_import__('datetime').datetime.now()}\n")
        print("Appended Successfully")

# New cart
new_cart = [("Biryani", 1, 180), ("Coke", 2, 100)]
append_order("Pooja", new_cart, 280)

Appended Successfully
```

```
with open("orders.txt", "r", encoding="utf-8") as f:
    print(f.read())

FOOD DELIVERY ORDER
Customer: Pooja - First Order
Burger x 2 = Rs.240
Total: Rs.240

--- NEW ORDER ---
Customer: Pooja
Biryani x 1 = Rs.180
Coke x 2 = Rs.100
Total: Rs.280
Time: 2026-09-21 11:04:38.283010
```

6.5 RENAME

```
# Rename
import os
with open("orders.txt", "w") as f:
    f.write("FOOD DELIVERY ORDER\nCustomer: Pooja\nTotal: Rs.490\n")
```

```
def rename_file():
    os.rename("orders.txt", "final_orders.txt")
    print("Renamed to final_orders.txt")
    # Check proof
    with open("final_orders.txt", "r") as f:
        print(f.read())

    os.rename("final_orders.txt", "orders.txt")
    print("\nRestored back to orders.txt")
```

```
rename_file()
```

```
Renamed to final_orders.txt
FOOD DELIVERY ORDER
Customer: Pooja
Total: Rs.490
```

6.6 DELETE

```
# Delete
import os
# Test file create pannu
with open("orders.txt", "w") as f:
    f.write("Test\n")
```

```
def delete_file():
    if os.path.exists("orders.txt"):
        os.remove("orders.txt")
        print("File Deleted Successfully")
    else:
        print("File not found")
```

```
delete_file()
```

```
File Deleted Successfully
```

```
print("File exists?", os.path.exists("orders.txt"))
if not os.path.exists("orders.txt"):
    print("Confirmed - Deleted")
```

```
File exists? False
Confirmed - Deleted
```

6.7 LIST

```
# List
with open("orders.txt", "w", encoding="utf-8") as f:
    f.write("FOOD DELIVERY ORDER\n")
    f.write("Customer: Pooja\n")
    f.write("Burger x 2 = Rs.240\n")
    f.write("Pizza x 1 = Rs.250\n")
    f.write("Total: Rs.490\n")
print("File ready")
```

```
File ready
```

```
def list_orders():
    with open("orders.txt", "r", encoding="utf-8") as file:
        lines = file.readlines()
        for i, line in enumerate(lines, 1):
            print(f"{i}. {line.strip()}")
```

```
list_orders()
```

```
1. FOOD DELIVERY ORDER
2. Customer: Pooja
3. Burger x 2 = Rs.240
4. Pizza x 1 = Rs.250
5. Total: Rs.490
```

6.7 SEARCH

```
def search_order():
    s = input("Enter name to search: ")
    with open("orders.txt", "r") as file:
        for line in file:
            if s.lower() in line.lower():
                print("Found ->", line.strip())
```

search_order()

Enter name to search: Pooja
Found -> Customer: Pooja

6.8 BACKUP

```
from datetime import datetime
def backup_orders():
    backup_name = f"orders_backup_{datetime.now().strftime('%d%m%Y')}.txt"
    with open("orders.txt", "r") as src, open(backup_name, "w") as dest:
        dest.write(src.read())
    print(f"Backup Created: {backup_name}")
```

backup_orders()

Backup Created: orders_backup_21092026.txt

6.9 COUNT

```
# Count
def count_orders():
    with open("orders.txt", "r") as file:
        count = len(file.readlines())
        print(f"Total lines: {count}")
```

count_orders()

Total lines: 5

6.10 HISTORY + LOG – RUN

```
# History + Log - Run
def read_orders():
    with open("orders.txt", "r") as f:
        print(f.read())

def show_history():
    print("----- ORDER HISTORY -----")
    read_orders()

from datetime import datetime
def log_activity(action):
    with open("activity_log.txt", "a") as log:
        log.write(f"{datetime.now()} - {action}\n")

# RUN
show_history()
log_activity("Login Success")
log_activity("Order Created for Pooja")
print("Log added")

# Check Log
with open("activity_log.txt", "r") as f:
    print(f.read())

----- ORDER HISTORY -----
FOOD DELIVERY ORDER
Customer: Pooja
Burger x 2 = Rs.240
Pizza x 1 = Rs.250
Total: Rs.490

Log added
2026-09-21 11:32:34.411965 - Login Success
2026-09-21 11:32:34.413802 - Order Created for Pooja
```

6.11 EXTENSIONS

```
# Extensions
cart = [("Burger", 2, 240), ("Pizza", 1, 250)]

# Lambda
square = lambda x: x**2
print(square(5)) # 25

# Map
totals = list(map(lambda x: x[2], cart))
print("Totals:", totals) # [240, 250]

# Filter
high = list(filter(lambda x: x>200, totals))
print("High bills:", high) # [240, 250]

# Generator
def even_bill_generator(cart):
    for item in cart:
        if item[2] % 2 == 0:
            yield item

for item in even_bill_generator(cart):
    print("Even:", item)

# Recursion
def factorial(n):
    return 1 if n <= 1 else n * factorial(n-1)

print("Fact 5:", factorial(5)) # 120

25
Totals: [240, 250]
High bills: [240, 250]
Even: ('Burger', 2, 240)
Even: ('Pizza', 1, 250)
Fact 5: 120
```

6.12 DUPLICATE

```
# duplicate check
def check_duplicate(cart):
    seen = set()
    for item in cart:
        if item[0] in seen:
            print(f"Duplicate: {item[0]}")
            return True
        seen.add(item[0])
    print("No duplicate")
    return False

# TEST 1 - No duplicate
cart1 = [("Burger", 2, 240), ("Pizza", 1, 250)]
check_duplicate(cart1) # No duplicate

# TEST 2 - Duplicate
cart2 = [("Burger", 2, 240), ("Burger", 1, 120)]
check_duplicate(cart2) # Duplicate: Burger

No duplicate
Duplicate: Burger
True
```

6.13 ACTIVITY LOG

```
# Activity Log
from datetime import datetime

def log_activity(action):
    with open("activity_log.txt", "a") as log:
        log.write(f"{datetime.now()} - {action}\n")
    print(f"Logged: {action}")

# RUN 3 Logs
log_activity("Login Success")
log_activity("Order Created for Pooja")
log_activity("File Deleted")

# Read Log file
with open("activity_log.txt", "r") as f:
    print("\n--- Activity Log ---\n", f.read())

Logged: Login Success
Logged: Order Created for Pooja
Logged: File Deleted

--- Activity Log ---
2026-09-21 11:32:34.411965 - Login Success
2026-09-21 11:32:34.413802 - Order Created for Pooja
2026-09-21 11:33:29.197364 - Login Success
2026-09-21 11:33:29.200920 - Order Created for Pooja
2026-09-21 11:33:29.202416 - File Deleted
```


CONCLUSION

The Food Delivery Order Management System, developed using Python file handling, successfully demonstrates the practical implementation of core programming concepts and file operations. Through this project, all major file modes — w (write), r (read), and a (append) — were implemented, along with OS module operations such as `rename()` and `remove()`. The system provides a complete workflow from user authentication (with a 3-attempt login limit) to order management, including creating, reading, appending, listing with line numbers, searching using the membership operator in, counting, backup creation, and deletion.

Additional Python concepts such as lambda functions, map, filter, generators with yield, recursion, and sets for duplicate detection were also integrated to enhance the functionality of the system. The activity log feature maintains a proper audit trail of all user actions with date and time using the datetime module. This project not only improves understanding of file handling but also gives experience similar to how a real food-delivery backend such as Swiggy or Zomato stores order data in files. The modular structure, with a separate function for each operation, makes the code reusable, readable, and easy to maintain.

In future, this project can be enhanced by adding a graphical interface using Tkinter, connecting to a database such as MySQL or SQLite, and implementing online payment and notification features. In conclusion, the project fulfils the objective of learning file handling in Python while building a useful, working application.

BIBLIOGRAPHY / REFERENCES

For the successful completion of this mini project, the following sources were referred to:

- 1 Python Official Documentation – docs.python.org/3/tutorial/inputoutput.html – referred for file handling modes (w, r, a).
- 2 W3Schools – Python File Handling – [w3schools.com/python/python_file_handling.asp](https://www.w3schools.com/python/python_file_handling.asp) – for open(), read(), readlines(), and enumerate().
- 3 GeeksforGeeks – Python OS Module – [geeksforgeeks.org/os-module-python](https://www.geeksforgeeks.org/os-module-python) – for os.rename(), os.remove(), and os.path.exists().
- 4 Programiz – Python Lambda, Map, Filter – [programiz.com/python-programming](https://www.programiz.com/python-programming) – for lambda, map, filter, and generators.
- 5 Textbook References – “Python Programming” by Dr. R. Nageswara Rao (File Handling chapter); “Let Us Python” by Yashavant Kanetkar (Functions and File Management).
- 6 YouTube Tutorials – CodeWithHarry's Python File Handling playlist; Telusko's Python Project Tutorials.
- 7 Class Notes – Gowthami Mam's lecture notes on file handling and mini project guidelines.